

Python Dictionaries — 25 Practice Questions

FAST-NUCES | BS Artificial Intelligence — Programming Fundamentals Practice Set

This set of 25 questions is arranged from basic to challenging and covers every major dictionary concept: creation, access, updating, deletion methods, membership testing, keys()/values()/items(), iteration patterns, counting, sorting, comprehensions, merging, nested dictionaries (2-level and 3-level), and real-world data-processing problems. Try to solve each question on your own before checking any reference material — no solutions are included here on purpose.

SECTION A — Fundamentals (Q1–Q6)

Q1. Basic — Creating a Dictionary

Create a dictionary named **student** that stores a student's *name*, *roll_number*, and *department*. Print the whole dictionary and also print its data type.

Concepts tested: Creating dictionaries, type()

Q2. Basic — Accessing Values Safely

You are given a dictionary of country–capital pairs. Ask the user to enter a country name and print its capital. If the country is not in the dictionary, print "Capital not found" instead of causing an error (use an appropriate dictionary method rather than square-bracket indexing).

```
capitals = {'Pakistan': 'Islamabad', 'Japan': 'Tokyo', 'France': 'Paris'}
```

Concepts tested: Accessing values, get() with default

Q3. Basic — Adding and Updating

Start with an empty dictionary called **employees**. Add five entries in the form `employee_id : name`. After adding all five, update the name stored against one existing `employee_id`, and add one brand-new key that did not exist before.

Concepts tested: Adding new keys, updating existing keys

Q4. Basic — Deletion Methods

Given the dictionary below, perform the following operations **in order** and print the dictionary after each step: (a) use **del** to remove 'Book C', (b) use **pop()** to remove 'Book A' and print the price that was returned, (c) use **popitem()** and print which item was removed, (d) finally use **clear()** and print the empty dictionary.

```
books = {'Book A': 850, 'Book B': 1200, 'Book C': 650, 'Book D': 999}
```

Concepts tested: del, pop(), popitem(), clear()

Q5. Basic — Checking Key Existence

Given a dictionary of usernames mapped to passwords, write code that asks the user to enter a username and prints "Username exists" or "Username not found" using the **in** keyword (do not use try/except or .get() for this one).

```
accounts = {'ali99': 'pass123', 'sara_k': 'qwerty', 'usman.b': 'abcd'}
```

Concepts tested: Membership testing with 'in'

Q6. Basic — keys(), values(), items(), len()

Given an inventory dictionary mapping item names to quantities, print (a) all the keys, (b) all the values, (c) all key-value pairs as tuples, and (d) the total number of items stored in the dictionary.

```
inventory = {'Pen': 120, 'Notebook': 45, 'Eraser': 200, 'Ruler': 60}
```

Concepts tested: keys(), values(), items(), len()

SECTION B — Loops, Conditions & Building Dictionaries (Q7–Q15)

Q7. Loop — Iterating with items()

Loop through a dictionary of products and their prices and print each one in the format "Product: , Price: Rs. ".

```
prices = {'Laptop': 145000, 'Mouse': 1200, 'Keyboard': 3500}
```

Concepts tested: for loop, items(), string formatting

Q8. Loop + Condition — Filtering While Looping

Given a dictionary of student names mapped to their marks (out of 100), loop through it and print the names of only those students who scored 80 or above.

```
marks = {'Ayesha': 92, 'Bilal': 76, 'Zara': 85, 'Hamza': 58, 'Noor': 81}
```

Concepts tested: for loop, conditional statement inside loop

Q9. Loop — Frequency / Counting Problem

Given a sentence as a string, build a dictionary that counts how many times each word appears in the sentence. Do not use collections.Counter — build the counting dictionary manually using a loop.

```
sentence = "the quick brown fox jumps over the lazy dog the fox runs"
```

Expected output hint: Expected (partial): {'the': 3, 'fox': 2, 'quick': 1, ...}

Concepts tested: Loop, dynamic dictionary building, frequency counting

Q10. Loop — Finding Maximum Manually

Given a dictionary mapping cricket players to the runs they scored, write a loop (do NOT use the built-in max() function directly on the dictionary) that finds and prints the player with the highest runs and their score.

```
runs = {'Babar': 87, 'Rizwan': 64, 'Shaheen': 12, 'Imam': 102, 'Fakhar': 45}
```

Concepts tested: Loop-based max search, comparison logic

Q11. Loop — Sorting Dictionary Data

Given a dictionary of employees and their salaries, create a list of (employee, salary) tuples sorted in descending order of salary, and print only the top 3 highest-paid employees.

```
salaries = {'Ahmed': 95000, 'Farah': 120000, 'Saad': 78000, 'Mahnoor': 135000, 'Bilal': 60000}
```

Concepts tested: Sorting (sorted() with key), slicing, tuples

Q12. Loop — Building a Dictionary Dynamically

You are given two separate lists: one of student names and one of their corresponding ages. Using a loop (not the zip() shortcut — build it manually with an index), construct a single dictionary that maps each name to its age.

```
names = ['Ali', 'Sana', 'Usman', 'Hira']  
ages = [21, 20, 22, 21]
```

Concepts tested: Dynamic dictionary creation, parallel lists, index-based loop

Q13. Dictionary Comprehension

Using a dictionary comprehension, create a dictionary that maps every number from 1 to 15 to its cube. Then, using a second dictionary comprehension (with a condition), create a new dictionary containing only the entries from the first one where the cube value is greater than 100.

Concepts tested: Dictionary comprehension, conditional comprehension

Q14. Multiple Dictionaries — Merging with a Rule

You are given two dictionaries representing the prices of the same products at Shop A and Shop B. Merge them into a single dictionary such that, for any product that appears in both shops, only the lower price is kept.

```
shopA = {'Sugar': 180, 'Rice': 320, 'Oil': 950}
shopB = {'Sugar': 175, 'Rice': 330, 'Flour': 140}
```

Concepts tested: Multiple dictionaries, merging, conditional logic

Q15. Comparing Two Dictionaries

Two dictionaries store a student's grade (as a percentage) in the mid-term and the final exam. Find and print the names of students who appear in both dictionaries AND whose final-exam percentage is higher than their mid-term percentage.

```
midterm = {'Ali': 70, 'Sana': 82, 'Zeb': 55, 'Imran': 90}
final = {'Ali': 78, 'Sana': 79, 'Zeb': 65, 'Owais': 88}
```

Concepts tested: Comparing dictionaries, common keys, conditional comparison

SECTION C — Nested Dictionaries & Real-World Problems (Q16–Q24)

Q16. Nested Dictionary — Computing Averages

You are given a nested dictionary where each student maps to another dictionary of their marks in three subjects. Loop through the outer dictionary and, for each student, compute and print their average mark.

```
results = {
    'Ayesha': {'Math': 88, 'Physics': 76, 'Chemistry': 91},
    'Bilal': {'Math': 65, 'Physics': 70, 'Chemistry': 60},
    'Zara': {'Math': 95, 'Physics': 89, 'Chemistry': 93}
}
```

Concepts tested: Nested dictionary, outer loop, aggregation (average)

Q17. Nested Dictionary — Search & Filter

You are given a nested dictionary of employees, where each employee maps to a dictionary containing their department, salary, and years of experience. Loop through it and print the names of all employees who work in the 'Sales' department AND earn more than 50,000.

```
employees = {
    'Hassan': {'department': 'Sales', 'salary': 62000, 'years': 3},
    'Neha': {'department': 'IT', 'salary': 75000, 'years': 5},
    'Kamran': {'department': 'Sales', 'salary': 41000, 'years': 1},
    'Ifra': {'department': 'Sales', 'salary': 58000, 'years': 4}
}
```

Concepts tested: Nested dictionary, filtering, multiple conditions, loop

Q18. Nested Dictionary — Nested Loop Aggregation

An inventory system stores products grouped by category, where each category maps to a dictionary of product:quantity pairs. Using a nested loop, calculate and print (a) the total number of distinct products across all categories, and (b) the total quantity of items held in each category.

```
inventory = {
    'Electronics': {'Mouse': 30, 'Keyboard': 15, 'Monitor': 8},
    'Stationery': {'Pen': 200, 'Notebook': 90},
    'Furniture': {'Chair': 12, 'Desk': 6, 'Shelf': 9}
}
```

Concepts tested: Nested dictionary, nested loop, aggregation per group

Q19. Nested Dictionary — Ranking with Nested Loops

A sports dictionary stores several teams, and each team maps to a dictionary of player:goals_scored. Using nested loops, find and print the top scorer WITHIN each team, and then determine the single top scorer across ALL teams combined.

```
teams = {
    'Falcons': {'Ahmed': 12, 'Zain': 9, 'Rehan': 15},
    'Tigers': {'Waleed': 20, 'Danish': 7},
    'Eagles': {'Fahad': 11, 'Junaid': 18, 'Salman': 5}
}
```

Concepts tested: Nested dictionary, nested loop, ranking, overall maximum

Q20. Three-Level Nested Dictionary

A university records its data as: university_department -> course -> list of enrolled student names (a three-level nested structure: dict -> dict -> list). Write nested loops to print, for every department, each course name along with the number of students enrolled in it, and also print the total number of students in the whole department.

```
university = {
    'Computer Science': {
        'AI': ['Ali', 'Zara', 'Hamid'],
        'Databases': ['Sana', 'Bilal']
    },
    'Electrical Engineering': {
        'Circuits': ['Neha', 'Imran', 'Owais', 'Faiza']
    }
}
```

Concepts tested: Three-level nested structure, multiple nested loops, counting

Q21. Nested Dictionary — Updating Records in a Loop

A bank stores accounts as a nested dictionary: account_number -> {name, balance}. Write a loop that goes through every account and applies a 5% interest increase to the balance of any account whose balance is currently above 10,000, updating the dictionary in place. Print the updated dictionary afterwards.

```
accounts = {
    101: {'name': 'Areeba', 'balance': 15000},
    102: {'name': 'Talha', 'balance': 8000},
    103: {'name': 'Mehak', 'balance': 22000}
}
```

Concepts tested: Nested dictionary, in-place update, condition inside loop

Q22. Nested Dictionary — Data Transformation

Using the nested student-marks dictionary from Question 16, create a BRAND-NEW dictionary that maps each student's name directly to a letter grade based on their average marks, using this scale: average ≥ 90 -> 'A', 80-89 -> 'B', 70-79 -> 'C', below 70 -> 'D'. The original nested dictionary should remain unchanged.

Expected output hint: Expected shape: {'Ayesha': 'B', 'Bilal': 'D', 'Zara': 'A'}

Concepts tested: Nested dictionary, data transformation, condition, creating a new dictionary

Q23. Nested Dictionary — Combining Two Structures

Two companies each store their department headcounts as a nested-style dictionary of department:employee_count. Combine the two dictionaries into one, so that for any department appearing in both companies, the employee counts are summed together, and departments unique to either company are kept as they are.

```
companyA = {'HR': 5, 'IT': 12, 'Sales': 8}
companyB = {'IT': 6, 'Sales': 4, 'Finance': 3}
```

Concepts tested: Merging dictionaries, combining data, loop-based summation

Q24. Nested Dictionary — Detecting Duplicates

An e-commerce system stores orders as: `order_id -> {customer, items}`. Using a loop, build a frequency dictionary that counts how many orders each customer has placed, then print the names of customers who have placed MORE THAN ONE order.

```
orders = {
  1: {'customer': 'Hania', 'items': ['Shoes', 'Bag']},
  2: {'customer': 'Danial', 'items': ['Watch']},
  3: {'customer': 'Hania', 'items': ['Jacket']},
  4: {'customer': 'Hania', 'items': ['Cap']},
  5: {'customer': 'Danial', 'items': ['Belt']}
}
```

Concepts tested: Nested dictionary, duplicate detection, frequency counting via loop

SECTION D — Final Challenge (Q25)

Q25. Comprehensive Real-World Challenge — Company Performance System

A company stores its data as a nested dictionary of the form **company -> department -> employee -> {salary, performance_rating}**. Using this data, write a program that performs ALL of the following tasks:

- 1 Calculate and print the **average salary** of each department.
- 2 Find and print the names of all employees, across every department, whose **performance_rating** is **4.5 or higher**.
- 3 Identify the department with the **lowest average salary** and give every employee in that department a **10% raise**, updating the original nested dictionary in place.
- 4 Build a brand-new summary dictionary in the form **department -> {avg_salary, top_employee}**, where **top_employee** is the name of the highest-rated employee in that department.

```
company = {
  'AI Solutions': {
    'Zohaib': {'salary': 90000, 'performance_rating': 4.8},
    'Rida': {'salary': 85000, 'performance_rating': 4.2}
  },
  'Cloud Services': {
    'Junaid': {'salary': 70000, 'performance_rating': 4.6},
    'Nimra': {'salary': 68000, 'performance_rating': 3.9},
    'Asadullah': {'salary': 72000, 'performance_rating': 4.5}
  },
  'Support': {
    'Farida': {'salary': 45000, 'performance_rating': 4.0},
    'Bakhtawar': {'salary': 47000, 'performance_rating': 3.5}
  }
}
```

Concepts tested: nested dictionaries, nested loops, conditional logic, aggregation (averages), searching/filtering, in-place updates, and creating a new derived dictionary — this question combines everything covered in the earlier 24 questions.

Tip: work through these in order. Questions 1–6 build your fundamentals, 7–15 build your loop and problem-solving muscle, and 16–25 train you on nested dictionaries — the pattern most commonly seen in real datasets and in FAST programming/AI coursework. Good luck!